

# PAPER\_502: WSTP Embedded Wolfram Kernel Bridge Architecture

*Unified Quantum Field Framework (UQFF)*

Daniel T. Murphy • UQFF Research Consortium • 2026

Source: PAPER\_502\_WSTP\_Embedded\_Kernel\_Bridge.md

---

## PAPER\_502: WSTP Embedded Wolfram Kernel Bridge Architecture

---

**Session:** 137 | **Source:** grok\_share\_84a767d3.txt (lines 300–700, 2300–3600) **Date:** November 2025 (first working commit: 8ae9ffe) **Related files:** source174\_wolfram\_bridge\_embedded.cpp

---

### §1.1 Abstract

---

The Wolfram Symbolic Transfer Protocol (WSTP) embedded kernel bridge provides a persistent, stateful connection between MAIN\_1\_CoAnQi.cpp and a live Wolfram Engine 14.3 process. This paper documents the canonical architecture, the five common failure modes, the correct launch string for the free Wolfram Engine, and the packet draining protocol that achieves reliable embedded operation.

---

### §1.2 Architecture Overview

---

The bridge uses three static pointers and four functions:

```
static WSENV ws_env = nullptr; // WSTP environment (one per process)
static WSLINK ws_link = nullptr; // Connection to kernel
```

**Function chain:**

```
main() → InitializeWolframKernel()
      ↓
      WSInitialize(nullptr) → ws_env
      ↓
      WSOpenArgv(ws_env, argv, &err) → ws_link
      ↓
      Drain startup packets (max 20)
```

```

      ↓
    atexit(WolframCleanup) registered
      ↓
WolframEvalToString(code) → packet exchange → result string
      ↓
WolframCleanup() → Exit + WSClose + WSDeinitialize

```

---

## §1.3 Canonical Launch String (Free Wolfram Engine 14.3)

---

```

const char* argv[] = {
    &quot;CoAnQi&quot;;, // dummy arg[0], ignored by WSTP
    &quot;;-linkmode&quot;;, &quot;;launch&quot;;, // WSTP creates the kernel as a child process
    &quot;;-linkname&quot;;,
    &quot;;\&quot;;C:\\Program Files\\Wolfram Research\\Wolfram Engine\\14.3\\wolfram.exe&quot;; -mathlink
    nullptr
};
int argc = 5;
int err = 0;
ws_link = WSOpenArgv(ws_env, argc, const_cast<char**>(argv), &err);

```

### Critical details:

- Use `wolfram.exe`, NOT `WolframKernel.exe` (free Engine requires wrapper)
  - `-mathlink` flag enables WSTP connection
  - `-nogui` suppresses "Connect to link:" dialog on Windows
  - The path must match installed Engine version (14.3 default shown)
- 

## §1.4 Packet Draining Protocol

---

After kernel launch, the kernel sends several startup packets before it is ready for evaluation. These must be drained:

```

int pkt, drain_count = 0;
while ((pkt = WSNextPacket(ws_link)) && pkt != RETURNPKT && drain_count < 20) {
    WSNewPacket(ws_link);
    drain_count++;
}
// Safety: if drain_count == 20, kernel may have sent unexpected packets

```

### Packet types encountered:

- `INPUTNAMEPKT` — kernel's In[n]:= prompt (discard)
  - `OUTPUTNAMEPKT` — kernel's Out[n]:= prompt (discard)
  - `RETURNPKT` — first actual response (stop draining)
-

## §1.5 String Result Evaluation Loop

```
std::string WolframEvalToString(const std::string& code) {
    WSPutFunction(ws_link, &quot;EvaluatePacket&quot;;, 1);
    WSPutFunction(ws_link, &quot;ToString&quot;;, 2);
    WSPutFunction(ws_link, &quot;FullSimplify&quot;;, 1);
    WSPutString(ws_link, code.c_str());
    WSPutSymbol(ws_link, &quot;InputForm&quot;);
    WSEndPacket(ws_link);
    WSFlush(ws_link);
    // Read RETURNPKT
    int pkt;
    while ((pkt = WSNextPacket(ws_link)) && pkt != RETURNPKT)
        WSNewPacket(ws_link);
    const char* result;
    WSGetString(ws_link, &result);
    std::string out(result);
    WSReleaseString(ws_link, result);
    return out;
}
```

## §1.6 Five Common Failure Modes

| # | Symptom                           | Root Cause                            | Fix                                                       |
|---|-----------------------------------|---------------------------------------|-----------------------------------------------------------|
| 1 | Silent timeout / "error: none"    | Wrong path (KernelExe vs wolfram.exe) | Change <code>wolfram.exe</code> to <code>-mathlink</code> |
| 2 | Linker error                      | Missing wstp64i4.lib                  | Add lib + WSTP include path to project                    |
| 3 | Kernel exits immediately          | Free Engine not activated             | Run <code>wolframscript -activate</code>                  |
| 4 | "Connect to link:" dialog         | Missing <code>-nogui</code> flag      | Append <code>-nogui</code> to launch string               |
| 5 | Hangs at "Waiting for WSActivate" | Listener mode (not direct)            | Switch to <code>-linkmode launch</code>                   |

## §1.7 Build Requirements

Compiler : MSVC v14.44+ (VS2022), x64 ONLY (WSTP is 64-bit)  
 Include path : C:\Program Files\Wolfram Research\Wolfram Engine\14.3\  
               SystemFiles\Links\WSTP\DeveloperKit\Windows-x86-64\  
               CompilerAdditions\wstp

```
Library      : wstp64i4.lib
Preprocessor  : USE_EMBEDDED_WOLFRAM
Standard     : C++20 (/std:c++20 /permissive-)
```

---

## §1.8 Equations

---

Connection quality metric:

$$Q_{\text{WSTP}} = \text{packets\_drained} / \text{max\_packets} \quad (\text{ideal: } Q \rightarrow 0)$$

Kernel latency estimate:

$$\tau_{\text{launch}} = t_{\text{ready}} - t_{\text{WSOpenArgv}} \quad (\text{empirical: 1-8 s, free Engine})$$

Evaluation throughput:

$$\eta_{\text{eval}} = N_{\text{terms}} / t_{\text{FullSimplify}} \quad (100\text{--}1000 \text{ terms/hour typical})$$

---

## §1.9 Citation

---

Source: grok\_share\_84a767d3.txt, lines 300–700, 2300–3600 Commit: 8ae9ffe — "Fix WSTP integration: wolfram.exe launch mode + fix infinite scan loop" Commit: 146a3c4 — "MSVC C++20 compatibility fixes and optimization enhancements" Paper number: PAPER\_502